ВНУТРЕННИЙ ДОКЛАД · [ДАТА]

Агентская разработка по спецификации

Как ИИ-агент за вечер написал клиент Redis для .NET и насколько хорошим получился результат

[Имя докладчика]



О ЧЁМ ДОКЛАД

План

1. **Что такое SDD и зачем**
   Проблема, идея, артефакты на примере реального проекта

2. **Результат**
   Цифры процесса и сравнение со StackExchange.Redis

3. **Живой прогон**
   Агент на быстрой модели исправляет найденный дефект у вас на глазах

4. **Критика**
   Где агент ошибся, чего не сделал и что с этим делать

ПРОБЛЕМА

Вайб-кодинг не масштабируется

Решения теряются

Они живут в чате и исчезают при сжатии контекста. Через неделю никто не помнит, почему сделано так.

Агент выбирает молча

Версия протокола, таймауты, семантика ошибок решаются где-то в середине диффа.

Ревью невозможно

Человеку приносят 15 000 строк сразу. Проверить можно только «вроде работает».

ИДЕЯ

Сначала договор, потом код

`proposal`

Зачем: проблема, объём, что не делаем

`design`

Как и почему так: решения и компромиссы

`specs`

Что должно выполняться: требования и сценарии

`tasks`

План: проверяемые шаги с чекбоксами

`apply`

Код и тесты строго по задачам

Человек ревьюит страницу текста, а не дифф. Все артефакты лежат в git рядом с кодом. Инструмент — OpenSpec, команды **`/opsx:new`**, **`/opsx:continue`**, **`/opsx:apply`**.

Proposal фиксирует рамки до первой строки кода

proposal.md

openspec › changes › archive › 2026-10-23-dotnet-redis-client › proposal.md

```
## Why

Нужен собственный Redis-клиент для .NET 10, написанный с нуля поверх протокола RESP, без зависимости от StackExchange.Redis. Клиент предназначен для enterprise-сервисов, поэтому он должен быть предсказуемым под нагрузкой, переживать сбои сети, работать по TLS с ACL-авторизацией и быть наблюдаемым штатными средствами .NET (логи, метрики, трассировка).
··· пропущено строк: 18 ···
### Вне рамок этого change

Каждая из этих возможностей станет отдельным change. Архитектура не должна закрывать дорогу к ним.
- Redis Cluster (топология слотов, `MOVED`/`ASK`).
- Redis Sentinel (обнаружение master и failover).
- Типизированные API для streams, geo, HyperLogLog, bitmaps и модулей (JSON, Search). До их появления эти команды доступны через `ExecuteAsync`.
- Client-side caching (`CLIENT TRACKING`).
- Синхронный (блокирующий) API.
```

Design: решения с альтернативами и ценой

design.md

openspec › changes › archive › 2026-10-23-dotnet-redis-client › design.md

```
38 ### D2. Модель конкурентности: мультиплексор с FIFO-очередью ожиданий
39
40 Одно физическое соединение обслуживает всех вызывающих.
41
42 - **Запись:** команды попадают в `Channel<PendingCommand>`. Единственный writer-loop вычитывает из канала всё, что
   накопилось, сериализует пачкой в `PipeWriter` и делает один `FlushAsync`. Так получается автоматический pipelining
   и coalescing.
43 - **Ожидания:** каждая записанная команда попадает в FIFO-очередь ожиданий со своим `TaskCompletionSource`.
44 - **Чтение:** единственный reader-loop парсит ответы и завершает ожидания строго по порядку. RESP3 push-сообщения
   не попадают в FIFO и маршрутизируются отдельно.
45 - **Число соединений:** опция `MultiplexedConnectionCount` (по умолчанию 1) задаёт N мультиплексированных
   соединений с round-robin. Это смягчает head-of-line blocking на больших значениях.
46 - **Альтернатива:** пул соединений «одна команда на соединение» (как в Npgsql). Её отвергли: нужно на порядки
   больше соединений к Redis, и pipelining теряется.
```

Там же в Risks записан риск **«ответы уходят не тем вызывающим»** и меры против него. Стенд: **0 перепутанных ответов.**

ПРИМЕР ИЗ ПРОЕКТА

Сценарий из спецификации становится тестом

SPECS / CONNECTION-MANAGEMENT

Scenario: Полуоткрытое соединение под нагрузкой

WHEN сервер перестал отвечать, TCP-соединение остаётся открытым, а приложение продолжает отправлять команды

THEN соединение пересоздаётся примерно через 3×CommandTimeout, не дожидаясь KeepAliveInterval

TIMEOUTANDBACKLOGTESTS.CS

```
[Fact]
public async Task
Stalled_connection_is_replaced_quickly()
```

Фейковый сервер воспроизводит условие из WHEN, тест проверяет поведение из THEN. Требование, тест и код ссылаются друг на друга.

Спецификация меняется вместе с кодом



план · 21:31

Нагрузочный тест заложен в tasks.md (задача 18.2) ещё до первой строки кода

тест · 22:49

Агент пишет его и гоняет 60 с: 64 потока, обрывы соединений, рестарты Redis

находка · 22:53

За docker-прокси клиент около минуты не замечает, что ответы не приходят. Исправление в коде

договор · 22:58

design.md: решение D10 с компромиссом. spec: новое требование и сценарий

Агент правит не только код, но и договор. Следующий читатель видит, что сделано, почему и какой ценой.

Что во всём этом делал человек

10

действий от идеи до готового клиента

`/opsx:new`

«давай сделаем клиента к редису на .net»

«1. с нуля 2. 10 3. реши сам … использоваться будет в кровавом энтерпрайзе»

`/opsx:continue ×3`

«давай 6.2» — минимальная версия Redis

«не нужно» — совместимость с Valkey и Dragonfly

Выбор имени пакета: Redis.Client

`/opsx:apply`

Что получилось за один вечер

22 мин

proposal, design, specs и tasks

~2 ч

реализация, 93 из 93 задач

8,8 тыс.

строк кода и 4,2 тыс. строк тестов

283

теста: 159 модульных и 124 на живом Redis 6.2 и 7.4

Плюс property-тесты парсера (2 500 случайных случаев), нагрузочный тест со сбоями, бенчмарки и CI.

ОБЪЁМ

Что внутри и чего нет

Есть

- RESP2 и RESP3, мультиплексирование, пул выделенных соединений
- Ключи, строки, хеши, списки, множества, sorted sets
- Транзакции с WATCH, Lua-скрипты
- Pub/Sub с восстановлением подписок
- TLS, ACL, OpenTelemetry, DI, NativeAOT

Нет, а в StackExchange.Redis есть

- Cluster и Sentinel
- Streams, Geo, HyperLogLog
- Кэширование на стороне клиента
- Префиксы ключей
- Годы работы в продакшене

САМОПРОВЕРКА

Ошибки, которые агент нашёл у себя сам

1

Переподключение умирало

Исключение при закрытии соединения молча останавливало цикл переподключения. Только на Redis 6.2, примерно в 1 прогоне из 7.

Нашёл нагрузочный тест, подтвердил регрессионный

2

Зависшие соединения

За docker-прокси клиент около минуты не замечал, что ответы не приходят.

Новое требование в spec, решение D10 в design

3

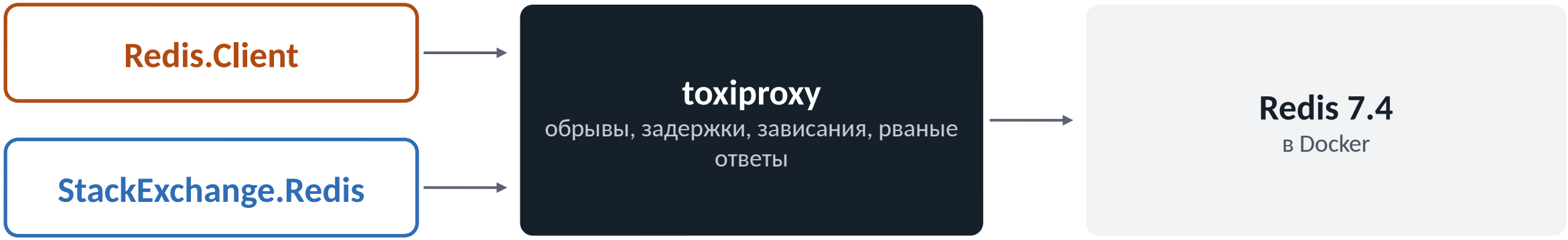
Квадратичный парсер

В собственном design был повторный разбор ответа с начала. Заменён на разбор с места остановки.

Массив из 100 элементов: 31,7 → 18,9 мкс

КАК СРАВНИВАЛИ

Стенд сравнения со StackExchange.Redis



Откуда

Агент написал его по одной просьбе: «давай подумаем, как сравнить с уже существующими решениями». Около получаса вместе с прогонами.

Что меряем

Пропускную способность, задержки p50/p99, CPU и аллокации. Шесть сценариев сбоев; оба клиента под одной нагрузкой со сверкой каждого ответа.

Как повторить

```
docker compose up -d
dotnet run -c Release -- all
```

Одинаковые таймауты, одно соединение у обоих.

СРАВНЕНИЕ · ПРОИЗВОДИТЕЛЬНОСТЬ

Тот же класс, что у StackExchange.Redis

Сценарий	Redis.Client	StackExchange.Redis
GET 100 Б, localhost	213 тыс. оп/с · 19,9 мкс CPU	176 тыс. оп/с · 26,6 мкс CPU
SET 100 Б, localhost	258 тыс. оп/с · 15,1 мкс CPU	211 тыс. оп/с · 22,2 мкс CPU
GET 100 Б, сеть +1 мс	93 тыс. оп/с · 15,4 мкс CPU	94 тыс. оп/с · 27,2 мкс CPU
p50 / p99, сеть +1 мс	2,0 / 3,4 мс	2,0 / 3,2 мс
GET 16 КБ, localhost	21,7 тыс. оп/с	22,1 тыс. оп/с
SET 16 КБ, localhost	31 тыс. оп/с — хуже на 42%	53 тыс. оп/с
Аллокации, 100 Б	0,8–0,9 КБ на операцию	0,4–0,6 КБ на операцию

Подсвечены строки, где наш клиент хуже.

256 вызывающих, у обоих одно соединение. Один прогон под WSL: сравнивать клиентов между собой, а не с продом.

СРАВНЕНИЕ · СБОИ

Никто не перепутал ни одного ответа

Сбой → время восстановления	Redis.Client	StackExchange.Redis
Обрыв соединений (TCP RST)	0,3 с	1,0 с
Сеть замерла на 10 с	0,2 с	2,0 с
Перезапуск Redis	0,7 с	1,2 с
Ответы приходят по 1 байту	без ошибок	без ошибок
Redis недоступен 10 с	7,8 с — дефект	2,1 с
Ответ не тому вызывающему	0	0
Самая долгая операция при таймауте 2 с	2,09 с	2,99 с

Подсвечена строка, где наш клиент хуже.

Оба клиента одновременно через один toxiproxy, по 32 потока: запись уникального значения и чтение со сверкой.

ЧАСТЬ 3

Живой прогон

Тот же процесс у вас на глазах, на быстрой модели Haiku. Две короткие сцены:

1

Начало проекта с нуля

`/opsx:new → proposal → design`

Тот же запрос «клиент к редису на .net»: как агент задаёт вопросы и фиксирует решения

2

Исправление в готовом коде

`/opsx:new fix-reconnect-backoff`

Стенд сравнения со StackExchange.Redis нашёл дефект. Исправим его тем же процессом и проверим замером

МАТЕРИАЛ ДЛЯ ДЕМО

Стенд нашёл дефект в нашей же спецификации

Восстановление после 10 с недоступности Redis

7,8 с — Redis.Client

2,1 с — StackExchange.Redis

`Паузы: 0,1 → 0,2 → 0,4 → … → 30 с`

Пауза между попытками переподключения растёт экспоненциально до 30 с. Так записано в spec: ReconnectMaxDelay = 30 с.

После минутного простоя клиент может «спать» до 30 с, когда сервер уже вернулся.

Свой нагрузочный тест агента нашёл зависание (слайд 8), а этот дефект пропустил: проверял, что клиент восстанавливается, но не как быстро. Нашло сравнение с чужим клиентом.

ЗАПАСНОЙ СЛАЙД · РЕПЕТИЦИЯ НА HAIKU

Что получилось у Haiku на репетиции

1

Ошибка в design

Предложил сбрасывать счётчик попыток после подключения, а он и так сбрасывается. Поймано на ревью страницы текста, до кода. После одной реплики — верное решение: пауза 30 с → 2 с.

2

Исправление работает

7,9 → 0,3 с

Восстановление после 10 с простоя Redis. У StackExchange.Redis — 2,0 с. Исправление — одна строка.

3

Отчёт ≠ реальность

Отмечены 27 из 27 задач, но трассировки нет и бенчмарк не запускался. Новый тест ни разу не запускался — и падает. README и spec всё ещё говорят «30 с».

Около 5 минут работы агента и $0,55. Вывод: ревьюить design до кода и проверять отчёт агента тестами и замером.

Где агент ошибся и чего не сделал

В spec не учтён граничный случай

После долгого простоя Redis клиент переподключается с опозданием. Нашёл стенд, а не тесты агента

SET 16 КБ медленнее на 42%

Причина пока не исследована

Аллокаций памяти больше

В 1,5–2 раза больше, чем у StackExchange.Redis, во всех сценариях

ОБСУЖДЕНИЕ

Что дальше?

1. Что из этого вы приняли бы на код-ревью от коллеги?
2. Где агенту нельзя доверять без человека?
3. Какие автопроверки обязательны, если код пишет агент?
4. Сколько времени это заняло бы у команды вручную?

Код, спецификации и стенд сравнения: github.com/IvanovJohn/redis.client